

1주차 예습과제

📖 딥러닝 파이토치 교과서 1, 2장

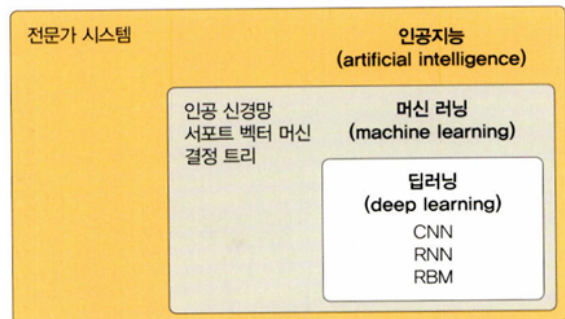
1장. 머신 러닝과 딥러닝

1.1 인공지능, 머신 러닝과 딥러닝

- 인공지능(Artificial Intelligence, AI)

- 인간의 지능을 모방하여 사람이 하는 일을 컴퓨터가 할 수 있도록 하는 기술
- 구현 방법으로 머신러닝과 딥러닝이 있음
- 인공지능 > 머신러닝 > 딥러닝

▼ 그림 1-1 인공지능과 머신 러닝, 딥러닝의 관계

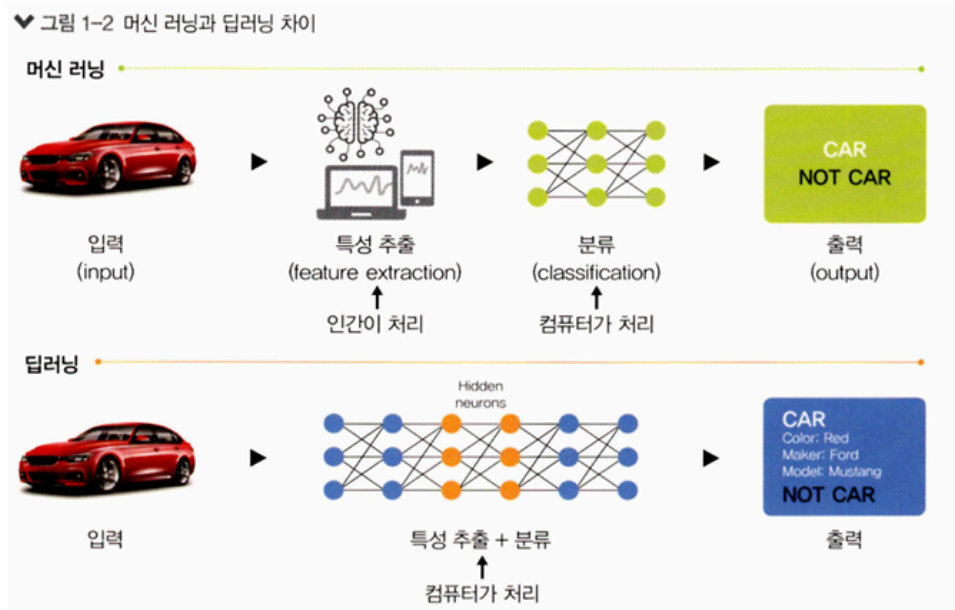


- 머신 러닝

- 주어진 데이터를 인간이 먼저 처리(전처리)
- 데이터의 특징을 스스로 추출하지 못하므로 이 과정을 인간이 처리해 주어야 함
- 각 데이터 특성을 컴퓨터에 인식, 학습시켜 문제 해결

- 딥러닝

- 인간이 하던 작업 생략
- 대량의 데이터를 신경망에 적용해 컴퓨터가 스스로 분석한 후 답을 찾음



구분	머신 러닝	딥러닝
동작 원리	입력 데이터에 알고리즘을 적용하여 예측을 수행	정보를 전달하는 신경망을 사용하여 데이터 특징 및 관계를 해석
재사용	입력 데이터를 분석하기 위해 다양한 알고리즘을 사용하며, 동일한 유형의 데이터 분석을 위한 재사용은 불가능	구현된 알고리즘은 동일한 유형의 데이터를 분석하는 데 재사용됨
데이터	일반적으로 수천 개의 데이터가 필요	수백만 개 이상의 데이터가 필요
훈련 시간	단시간	장시간
결과	일반적으로 점수 또는 분류 등 숫자 값	출력은 점수, 텍스트, 소리 등 어떤 것이든 가능

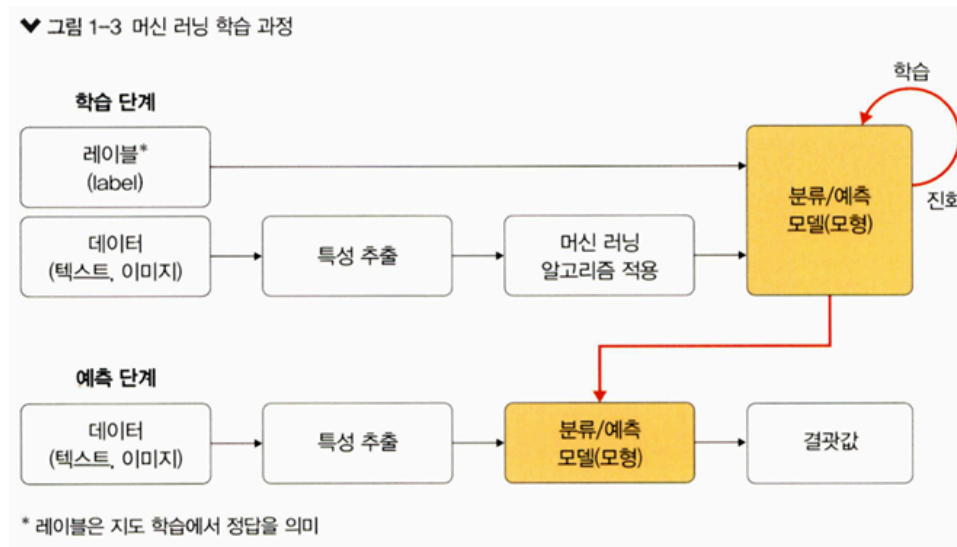
1.2 머신 러닝이란

- 머신 러닝은 인공지능의 한 분야로, 컴퓨터 스스로 대용량 데이터에서 지식이나 패턴을 찾아 학습하고 예측을 수행하는 것
- 즉, 컴퓨터가 학습할 수 있게 하는 알고리즘과 기술을 개발하는 분야

1.2.1 머신 러닝 학습 과정

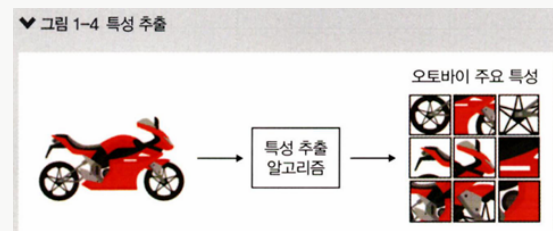
- 크게 학습 단계(learning)와 예측 단계(prediction)로 구분
- 학습 단계: 훈련 데이터를 머신 러닝 알고리즘에 적용하여 학습시키고, 이 학습 결과로 모형이 생성됨

- 예측 단계: 학습 단계에서 생성된 모형에 새로운 데이터를 적용하여 결과를 예측



▼ 특성 추출(feature extraction)

- 머신 러닝에서 컴퓨터가 스스로 학습하려면, 즉 컴퓨터가 입력받은 데이터를 분석하여 일정 패턴이나 규칙을 찾아내려면 사람이 인지하는 데이터를 컴퓨터가 인지할 수 있는 데이터로 변환해 주어야 함
- 특성 추출: 데이터별로 어떤 특징을 가지고 있는지 찾아내고, 그것을 토대로 데이터를 벡터로 변환하는 작업

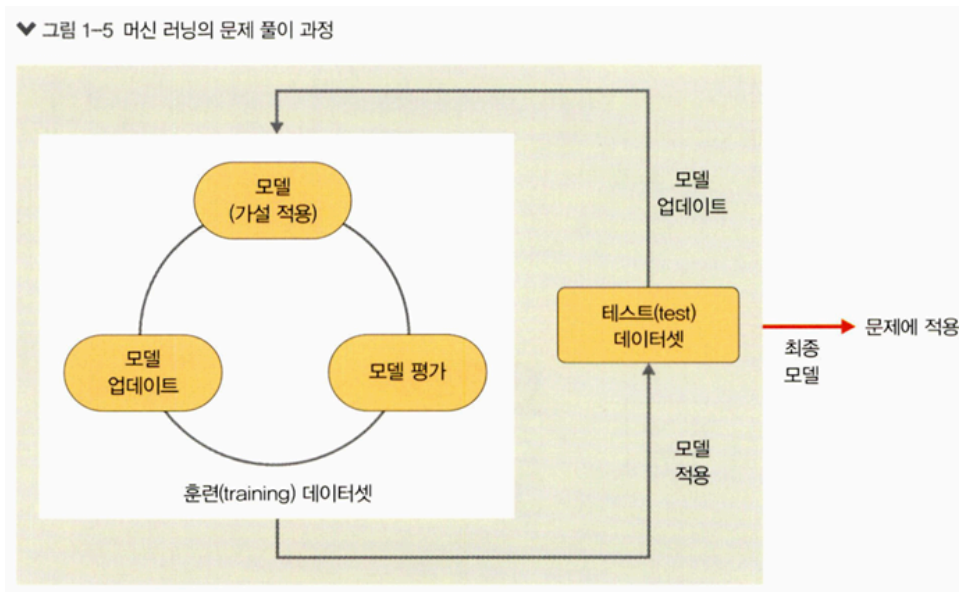


머신러닝의 주요 구성 요소

- 데이터
 - 머신 러닝이 학습 모델을 만드는 데 사용하는 것
 - 훈련 데이터가 나쁘다면 실제 현상의 특성을 제대로 반영할 수 없음
 - 따라서 실제 데이터의 특징이 잘 반영되고 편향되지 않는 훈련 데이터를 확보하는 것이 중요
 - 수집한 데이터는 '훈련 데이터셋'과 '테스트 데이터셋' 용도로 분리해서 사용, '훈련 데이터셋'을 또다시 '훈련 데이터셋'과 '검증 데이터셋'으로 분리해서 사용 (보통 8:2 비율)

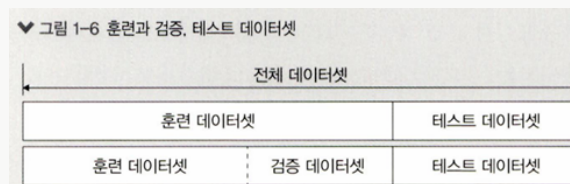
• 모델(모형)

- 머신 러닝의 학습 단계에서 얻은 최종 결과물, 가설이라고도 함
- ex) "입력 데이터의 패턴은 A와 같다." 라는 가정
- 학습 절차
 1. 모델(또는 가설) 선택
 2. 모델 학습 및 평가
 3. 평가를 바탕으로 모델 업데이트



- 세 단계를 반복해 주어진 문제를 가장 잘 풀 수 있는 모델을 찾고, 최종적으로 완성된 모델을 해결하고자 하는 문제에 적용해서 분류 및 예측 결과를 도출

▼ 훈련과 검증, 테스트 데이터셋



- 모델의 성능을 평가하기 위해서 검증 데이터셋을 사용
- 검증 데이터셋은 훈련 데이터셋의 일부를 떼어서 사용하기 때문에 학습에 사용되는 데이터셋의 양이 많지 않다면 검증 데이터셋을 사용하는 것은 좋지 않음
- 모델 성능의 평가가 필요한 이유

1. 테스트 데이터셋에 대한 성능을 짐작할 수 있음
2. 모델 성능을 높이는 데 도움을 줌

1.2.2 머신 러닝 학습 알고리즘

• 지도 학습

- 정답이 무엇인지 컴퓨터에 알려 주고 학습시키는 방법

• 비지도 학습

- 정답을 알려 주지 않고 특징이 비슷한 데이터를 클러스터링(범주화)하여 예측하는 학습 방법

▼ 그림 1-8 지도 학습과 비지도 학습



- 지도 학습은 주어진 데이터에 대해 A 혹은 B로 명확한 분류가 가능하지만, 비지도 학습은 유사도 기반(데이터 간 거리 측정)으로 특징이 유사한 데이터끼리 클러스터링으로 묶어서 분류

• 강화 학습

- 자신의 행동에 대한 보상을 받으며 학습을 진행
- ex) 게임 → 에이전트가 변화하는 환경에 따라 다른 행동을 취하게 되고, 행동에 따라 보상을 얻는데 이런 보상이 커지는 행동은 자주 하도록 하고 보상이 줄어드는 행동은 덜 하도록 하여 학습을 진행

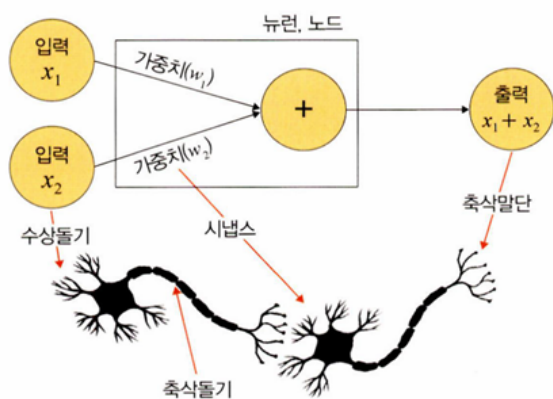
구분	유형	알고리즘
지도 학습 (supervised learning)	분류(classification)	K-최근접 이웃(K-Nearest Neighbor, KNN) 서포트 벡터 머신(Support Vector Machine, SVM) 결정 트리(decision tree) 로지스틱 회귀(logistic regression)
	회귀(regression)	선형 회귀(linear regression)

비지도 학습 (unsupervised learning)	군집(clustering)	K-평균 군집화(K-means clustering) 밀도 기반 군집 분석(DBSCAN)
	차원 축소 (dimensionality reduction)	주성분 분석(Principal Component Analysis, PCA)
강화 학습 (reinforcement learning)	-	마르코프 결정 과정(Markov Decision Process, MDP)

1.3 딥러닝이란

- 인간의 신경망 원리를 모방한 심층 신경망 이론을 기반으로 고안된 머신 러닝 방법의 일종
- 인간의 뇌를 기초로 하여 설계했다는 것이 머신 러닝과 다른 큰 차이점
- 인간의 뇌가 엄청난 수의 뉴런(neuron)과 시냅스(synapse)로 구성되어 있는 것에 착안하여 컴퓨터에 뉴런과 시냅스 개념 적용
- 각각의 뉴런은 복잡하게 연결된 수많은 뉴런을 병렬 연산하여 기존에 컴퓨터가 수행하지 못했던 음성·영상 인식 등의 처리를 가능하게 함

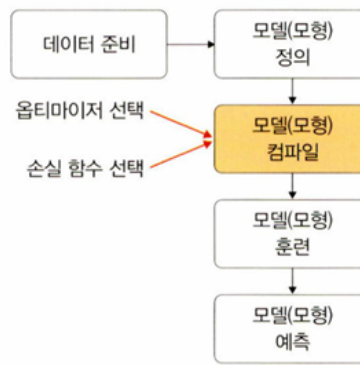
▼ 그림 1-10 인간의 신경망 원리를 모방한 심층 신경망



- 수상돌기: 주변이나 다른 뉴런에서 자극을 받아들이고, 이 자극들을 전기적 신호 형태로 세포체와 축삭돌기로 보내는 역할
- 시냅스: 신경 세포들이 이루는 연결 부위로, 한 뉴런의 축삭돌기와 다음 뉴런의 수상돌기가 만나는 부분
- 축삭돌기: 다른 뉴런(수상돌기)에 신호를 전달하는 기능을 하는 뉴런의 한 부분
- 축삭말단: 전달된 전기 신호를 받아 신경 전달 물질을 시냅스 틈새로 방출

1.3.1 딥러닝 학습 과정

▼ 그림 1-11 딥러닝 모델의 학습 과정



• 데이터 준비

- 파이토치Pytorch나 케라스Keras에서 제공하는 데이터셋 사용
 - 이미 전처리를 해서 제공되는 데이터이기 때문에 바로 사용할 수 있으며, 수많은 예제 코드를 쉽게 구할 수 있는 장점
- 캐글Kaggle 같은 곳에 공개된 데이터를 사용

• 모델(모형) 정의

- 모델 정의 단계에서 신경망을 생성
- 일반적으로 은닉층 개수가 많을수록 성능이 좋아지지만 과적합이 발생할 확률 높아짐

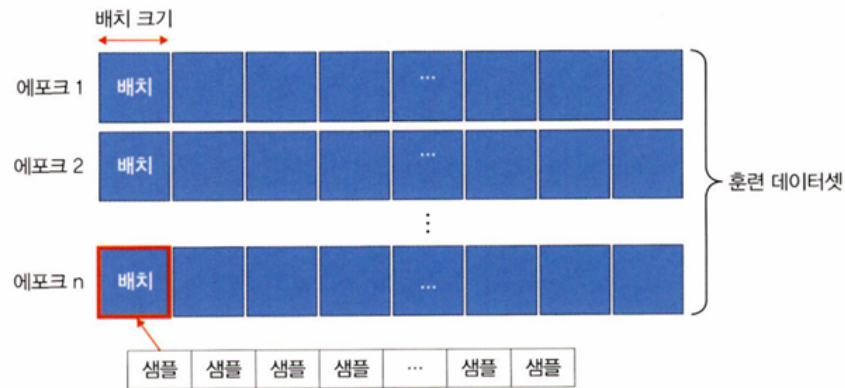
• 모델(모형) 컴파일

- 컴파일 단계에서 활성화 함수, 손실 함수, 옵티마이저를 선택
- ex) 훈련 데이터셋 형태가 연속형이라면 평균 제곱 오차(Mean Squared Error, MSE) 사용, 이진 분류(binary classification)이라면 크로스 엔트로피(cross entropy) 선택
- 과적합을 피할 수 있는 활성화 함수 및 옵티마이저 선택이 중요

• 모델(모형) 훈련

- 훈련 단계에서 한 번에 처리할 데이터양을 지정
- 한 번에 처리해야 할 데이터양이 많아지면 학습 속도가 느려지고 메모리 부족 문제를 야기할 수 있으므로 적당한 데이터양을 선택하는 것이 중요
- 전체 훈련 데이터셋에서 일정한 묶음으로 나누어 처리할 수 있는 배치와 훈련의 횟수인 에포크 선택이 중요
- 훈련 과정에서 값의 변화를 시각적으로 표현하여 눈으로 확인하면서 파라미터와 하이퍼파라미터에 대한 최적의 값 찾을 수 있어야 함

▼ 그림 1-12 모델 훈련에 필요한 하이퍼파라미터



- ex) '훈련 데이터셋 1000개에 대한 배치 크기가 20' → 샘플 단위 20개마다 모델 가중치를 한 번씩 업데이트시킨다 → 총 50번의 가중치가 업데이트됨
이때 에포크가 10이고 배치 크기가 20 → 가중치를 50번 업데이트하는 것을 총 10번 반복 → 결과적으로 가중치가 총 500번 업데이트 됨

▼ 성능이 좋다는 의미는?

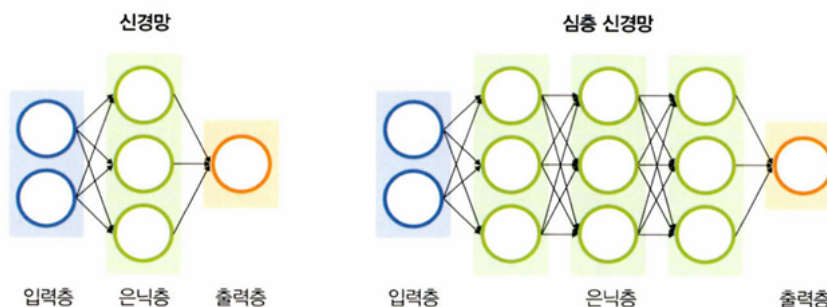
- 머신 러닝/딥러닝에서 성능(performance)에 대한 공식적인 정의는 없음
- 예측을 잘한다(정확도가 높다), 훈련 속도가 빠르다 등의 다양한 의미

• 모델(모형) 예측

- 검증 데이터셋을 생성한 모델(모형)에 적용하여 실제로 예측을 진행해보는 단계
- 예측력이 낮다면 파라미터를 튜닝하거나 신경망 자체를 재설계해야 할 수 있음

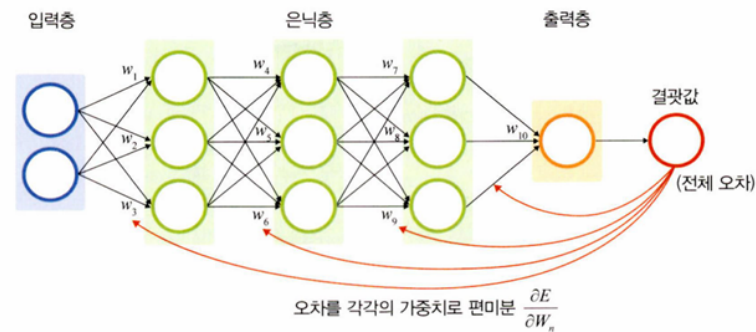
- 딥러닝 학습 과정에서 가장 중요한 핵심 구성 요소: 신경망과 역전파
- 딥러닝은 심층 신경망(deep neural network, 은닉층이 두 개 이상인 신경망)을 사용한다는 점에서 머신 러닝과 차이가 있음
- 심층 신경망에는 데이터셋의 어떤 특성들이 중요한지 스스로에게 가르쳐줄 수 있는 기능 있음

▼ 그림 1-13 신경망과 심층 신경망



- 가중치 값을 업데이트 하기 위한 역전파가 중요
- 역전파 계산 과정에서 사용되는 미분(오차를 각 가중치로 미분)이 성능에 영향을 미치는 주요한 요소

♥ 그림 1-14 역전파 계산



- 파이토치와 같은 프레임워크를 이용하면 역전파 알고리즘을 자동으로 처리해 주기 때문에 딥러닝 알고리즘 구현이 간단하고 편리

1.3.2 딥러닝 학습 알고리즘

- 활용 분야에 따라 지도 학습, 비지도 학습, 전이 학습으로 분류
- 지도 학습
 - 이미지 분류
 - 이미지 또는 비디오상의 객체를 식별하는 컴퓨터 비전 기술
 - 합성곱 신경망(Convolutional Neural Network, CNN)
 - 목적에 따라 이미지 분류, 이미지 인식, 이미지 분할로 분류
 - 이미지 분류: 이미지 데이터를 유사한 것끼리 분류
 - 이미지 인식: 사진을 분석하여 그 안에 있는 사물의 종류를 인식
 - 이미지 분할: 영상에서 사물이나 배경 등 객체 간 영역을 픽셀 단위로 구분

♥ 그림 1-15 이미지 인식출처: https://www.researchgate.net/figure/Object-detection-in-a-dense-scene_fig4_329217107



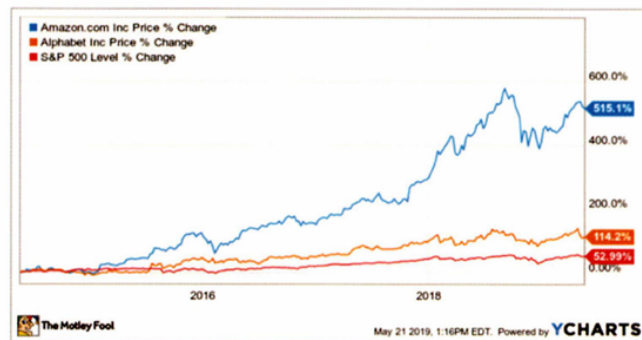
○ 시계열 데이터 분석

■ 순환 신경망(Recurrent Neural Network, RNN)

- 시간에 따른 데이터가 있을 때 사용
- 역전파 과정에서 기울기 소멸 문제가 발생하는 단점
- 게이트(gate)를 세 개 추가해 문제점 개선: LSTM(Long Short-Term Memory)
 - 망각 게이트(과거 정보를 잊기 위한 게이트), 입력 게이트(현재 정보를 기억하기 위한 게이트), 출력 게이트(최종 결과를 위한 게이트)를 도입
 - 현재 시계열 문제에서 가장 활발히 사용하고 있음

♥ 그림 1-16 구글과 아마존 주식에 대한 시계열 데이터 사례

(출처: <https://www.foo.com/investing/2019/05/26/better-buy-amazon-vs-google.aspx>)



● 비지도 학습

○ 워드 임베딩(word embedding)

- 자연어(사람의 언어)를 컴퓨터가 이해하고 효율적으로 처리하게 하기 위해 컴퓨터가 이해할 수 있도록 적절히 변환 필요
- 워드 임베딩 기술을 이용하여 단어를 벡터로 표현
- 자연어 처리 분야의 일종으로 번역이나 음성 인식 등의 서비스에서 사용

- 풀고자 하는 문제와 비슷하면서도 많은 데이터로 이미 학습이 되어 있는 모델
- VGG, 인셉션Inception, MobileNet 같은 사전 학습 모델 사용하면 효율적인 학습이 가능
- 활용하는 방법으로 특성 추출과 미세 조정 기법이 있음

• 강화 학습

- 머신 러닝과 동일

구분	유형	알고리즘
지도 학습 (supervised learning)	이미지 분류	CNN AlexNet ResNet
	시계열 데이터 분석	RNN LSTM
비지도 학습 (unsupervised learning)	군집(clustering)	가우시안 혼합 모델(Gaussian Mixture Model, GMM) 자기 조직화 지도(Self-Organizing Map, SOM)
	차원 축소	오토인코더(AutoEncoder) 주성분 분석(PCA)
전이 학습 (transfer learning)	전이 학습	버트(BERT) MobileNetV2
강화 학습 (reinforcement learning)	-	마르코프 결정 과정(MDP)

2장. 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

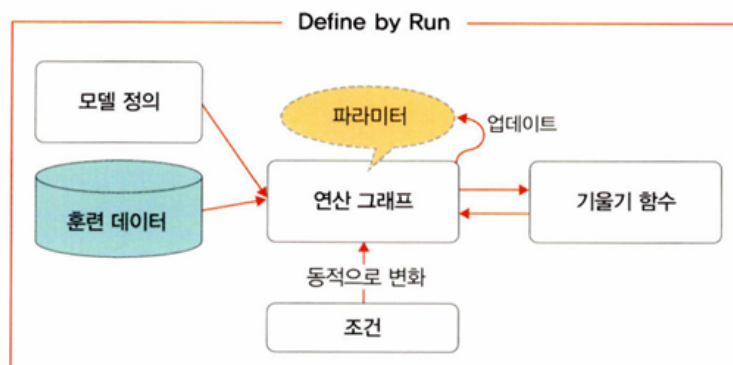
- 파이토치Pytorch는 2017년 초에 공개된 딥러닝 프레임워크로, 루아Lua 언어로 개발되었던 토치Torch를 페이스북에서 파이썬 버전으로 내놓음
- 넘파이Numpy를 대체하면서 GPU를 이용한 연산이 필요한 경우
- 최대한의 유연성과 속도를 제공하는 딥러닝 연구 플랫폼이 필요한 경우

- 간결하고 빠른 구현성

2.1.1 파이토치 특징 및 장점

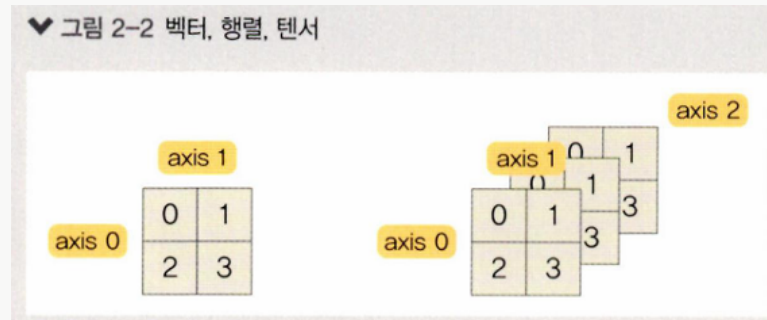
- GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크
- GPU(Graphics Processing Unit)
 - 연산 속도를 빠르게 하는 역할
 - 딥러닝에서는 기울기를 계산할 때 미분을 쓰는데, GPU를 사용하면 빠른 계산이 가능
 - 내부적으로 CUDA, cuDNN이라는 API를 통해 GPU를 연산에 사용할 수 있음
 - 병렬 연산에서 GPU의 속도는 CPU의 속도보다 훨씬 빠르므로 딥러닝 학습에서 GPU 사용은 필수
- 텐서(Tensor)
 - 파이토치의 데이터 형태
 - 단일 데이터 형식으로 된 자료들의 다차원 행렬
 - 간단한 명령어(변수 뒤에 `.cuda()` 를 추가)를 사용해서 GPU로 연산을 수행하게 할 수 있음
- 동적 신경망
 - 훈련을 반복할 때마다 네트워크 변경이 가능한 신경망을 의미
 - 학습 중에 은닉층을 추가하거나 제거하는 등 모델의 네트워크 조작이 가능
 - 연산 그래프를 정의하는 것과 동시에 값도 초기화되는 'Define by Run' 방식을 사용, 연산 그래프와 연산을 분리해서 생각할 필요가 없기 때문에 코드를 이해하기 쉬움

▼ 그림 2-1 파이토치 'Define by Run'



▼ 벡터, 행렬, 텐서

- 인공지능에서 데이터는 벡터(vector)로 표현됨
- 벡터: 숫자들의 리스트, 1차원 배열 형태 → 1차원 축(행)=axis 0=벡터
- 행렬(matrix): 행(row, 가로줄)과 열(column, 세로줄)로 표현되는 2차원 배열 형태 → 2차원 축(열)=axis 1=행렬
- 텐서: 3차원 이상의 배열 형태 → 3차원 축(채널)=axis 2=텐서



- 행렬은 복수의 차원을 가지는 데이터 레코드의 집합
- 하나의 데이터 레코드를 벡터 단독으로 나타낼 때는 하나의 열로 표기
- 복수의 데이터 레코드 집합을 행렬로 나타낼 때는 하나의 데이터 레코드가 하나의 행으로 표기
- 텐서는 행렬의 다차원 표현으로, 같은 크기의 행렬이 여러 개 묶여 있는 것
- 파이토치에서 텐서를 표현하기 위해서는 `torch.tensor()` 를 사용

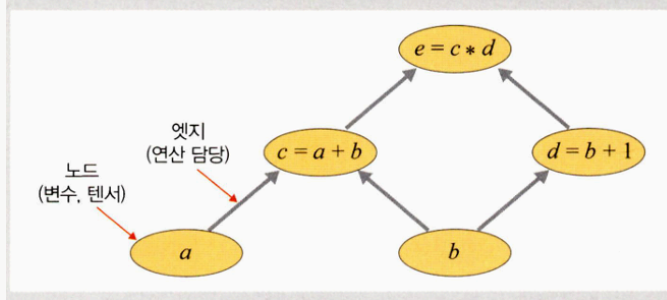
```
import torch
torch.tensor([[1., -1.], [1., -1.]])

# 생성된 텐서의 형태는 다음과 같이 표현됨
tensor([[ 1., -1.],
        [ 1., -1.]])
```

▼ 연산 그래프

- 연산 그래프는 방향성이 있으며 변수를 의미하는 노드와 연산을 담당하는 엣지로 구성됨

♥ 그림 2-3 파이토치 연산 그래프



- 노드는 변수(a, b)를 가지고 있으며 각 계산을 통해 새로운 텐서(c, d, e)를 구성할 수 있음
- 신경망은 연산 그래프를 이용하여 계산을 수행
- 네트워크가 학습될 때 손실 함수의 기울기가 가중치와 바이어스를 기반으로 계산, 이후 경사 하강법을 사용하여 가중치가 업데이트 됨

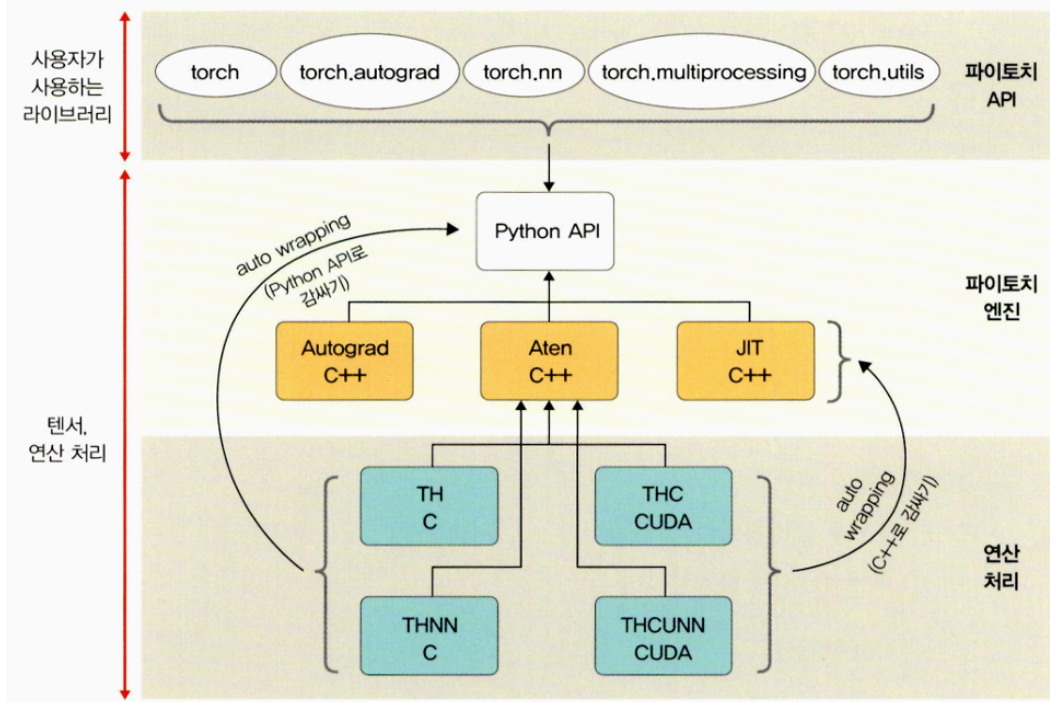
파이토치의 장점

- 단순함(효율적인 계산)
 - 파이썬 환경과 쉽게 통합할 수 있음
 - 디버깅이 직관적이고 간결
- 성능(낮은 CPU 활용)
 - 모델 훈련을 위한 CPU 사용이 텐서플로와 비교하여 낮음
 - 학습 및 추론 속도가 빠르고 다루기 쉬움
- 직관적인 인터페이스
 - 텐서플로처럼 잦은 API 변경이 없어 배우기 쉬움

2.1.2 파이토치의 아키텍처

- 파이토치의 아키텍처는 크게 세 개의 계층으로 나누어 설명할 수 있음
- 가장 상위 계층은 파이토치 API가 위치, 그 아래에는 다차원 텐서 및 자동 미분을 처리하는 파이토치 엔진이 있음
- 가장 아래에는 텐서에 대한 연산을 처리하며 CPU/GPU를 이용하는 텐서의 실질적인 계산을 위한 C, CUDA 등 라이브러리가 위치

▼ 그림 2-4 파이토치의 아키텍처



파이토치 API

- 사용자가 이해하기 쉬운 API를 제공하여 텐서에 대한 처리와 신경망을 구출하고 훈련할 수 있도록 도움
- 사용자 인터페이스를 제공하지만 실제 계산은 수행하지 않음, C++로 작성된 파이토치 엔진으로 그 작업을 전달하는 역할만 함
- 제공 패키지
 - **torch: GPU를 지원하는 텐서 패키지**
 - 다차원 텐서를 기반으로 다양한 수학적 연산이 가능하도록 함
 - GPU에서 연산이 가능하므로 빠른 속도로 많은 양의 계산 할 수 있음
 - **torch.autograd: 자동 미분 패키지**
 - Autograd는 텐서플로TensorFlow, 카페Caffe, CNTK 같은 다른 딥러닝 네트워크와 가장 차별되는 패키지
 - 일반적으로 신경망에 사소한 변경(ex-은닉층 노드 수 변경)이 있다면 신경망 구축을 처음부터 다시 시작해야 하지만 파이토치는 '자동 미분(auto-differentiation)'이라는 기술을 채택하여 미분 계산을 효율적으로 처리
 - 연산 그래프가 즉시 계산(실시간으로 네트워크 수정이 반영된 계산)되기 때문에 사용자는 다양한 신경망을 적용해 볼 수 있음

- **torch.nn: 신경망 구축 및 훈련 패키지**

- `torch.nn` 을 사용할 경우 신경망을 쉽게 구축하고 사용할 수 있음
- 합성곱 신경망, 순환 신경망, 정규화 등이 포함되어 손쉽게 신경망을 구축하고 학습시킬 수 있음

- **torch multiprocessing: 파이썬 멀티프로세싱 패키지**

- 파이토치에서 사용하는 프로세스 전반에 걸쳐 텐서의 메모리 공유가 가능
- 따라서 서로 다른 프로세스에서 동일한 데이터(텐서)에 대한 접근 및 사용이 가능

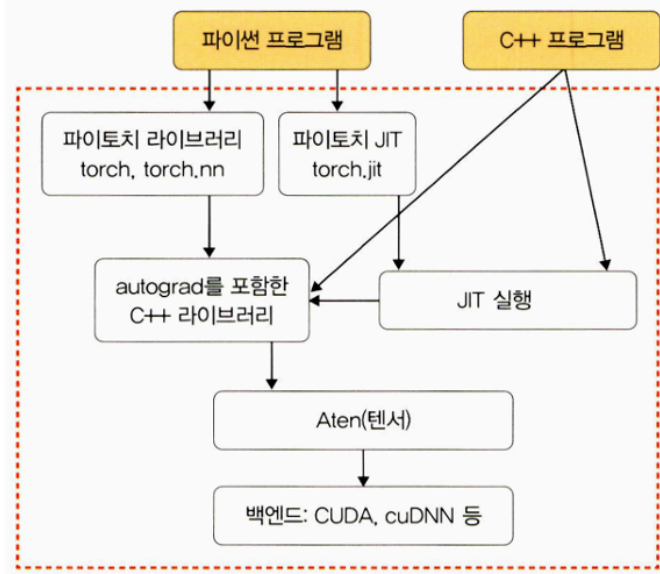
- **torch.utils: DataLoader 및 기타 유틸리티를 제공하는 패키지**

- 모델에 데이터를 제공하기 위한 `torch.utils.data.DataLoader` 모듈을 주로 사용
- 병목 현상을 디버깅하기 위한 `torch.utils.bottleneck` , 모델 또는 모델의 일부를 검사하기 위한 `torch.utils.checkpoint` 등의 모듈도 있음

파이토치 엔진

- Autograd C++, Aten C++, JIT C++, Python API로 구성
- Autograd C++: 가중치, 바이어스를 업데이트하는 과정에서 필요한 미분을 자동으로 계산
- Aten C++: C++ 텐서 라이브러리 제공
- JIT C++: 계산을 최적화하기 위한 JIT(Just In-Time) 컴파일러
- 파이토치 엔진 라이브러리는 C++로 감싼(래핑, wrapping) 다음 Python API 형태로 제공되기 때문에 사용자들이 손쉽게 모델을 구축하고 텐서를 사용할 수 있음

▼ 그림 2-5 파이토치 엔진



연산 처리

- C 또는 CUDA 패키지는 상위의 API에서 할당된 거의 모든 계산을 수행
- 여기서 제공되는 패키지는 CPU와 GPU(TH(토치), THC(토치 CUDA))를 이용하여 효율적인 데이터 구조, 다차원 텐서에 대한 연산을 처리

▼ 텐서를 메모리에 저장하기

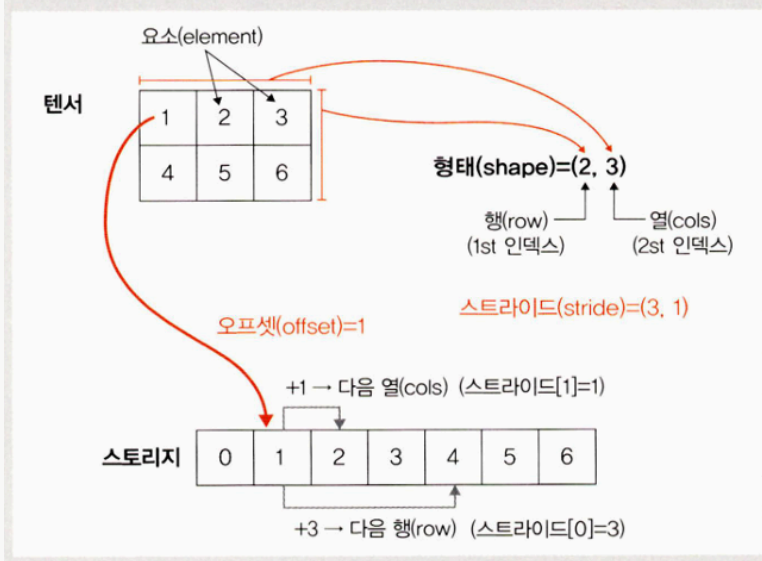
- 텐서는 1차원 배열 형태여야만 메모리에 저장할 수 있음
- 변환된 1차원 배열을 스토리지(storage)라고 함
- 오프셋(offset): 텐서에서 첫 번째 요소가 스토리지에 저장된 인덱스
- 스트라이드(stride): 각 차원에 따라 다음 요소를 얻기 위해 건너뛰기(skip)가 필요한 스토리지의 요소 개수 → 메모리에서 텐서 레이아웃을 표현하는 것
- 요소가 연속적으로 저장되기 때문에 행 중심으로 스트라이드는 항상 1

▼ 그림 2-6 3차원 텐서를 1차원으로 변환



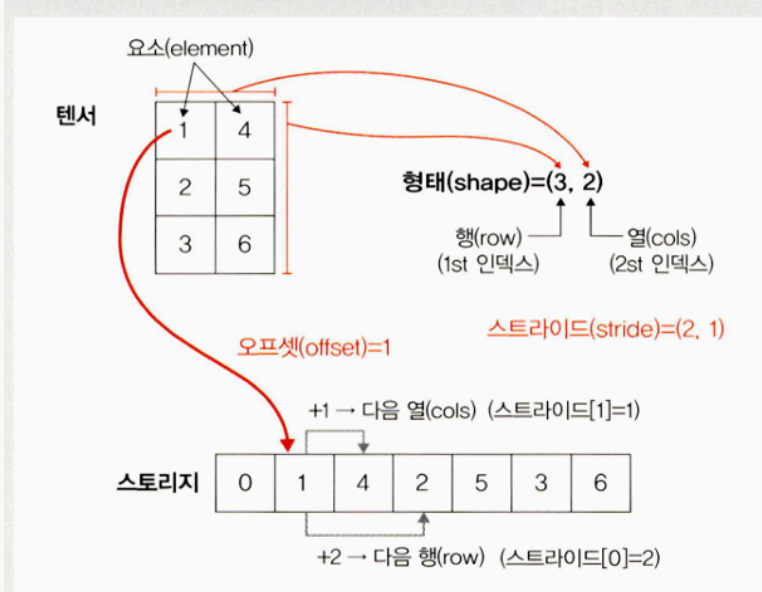
- A 와 A^T 는 다른 형태를 갖지만 스토리지의 값들은 서로 같다 → A 와 A^T 를 구분하는 용도로 오프셋과 스트라이드를 사용

▼ 그림 2-7 A를 1차원으로 변환



- A 의 스토리지에서 2를 얻기 위해서는 1에서 1칸을 뛰어넘어야 하고, 4를 얻기 위해서는 3을 뛰어넘어야 하므로 텐서에 대한 스토리지의 스트라이드는 (3, 1)

▼ 그림 2-8 A의 전치 행렬을 1차원으로 변환



- A^T 의 스토리지에서 4를 얻기 위해서는 1에서 1칸을 뛰어넘어야 하고, 2를 얻기 위해서는 2를 뛰어넘어야 하므로 텐서에 대한 스토리지의 스트라이드는 (2, 1)
- 이와 같이 오프셋과 스트라이드는 데이터 자체가 아닌 행렬/텐서를 구분하기 위해 사용

2.2 파이토치 기초 문법

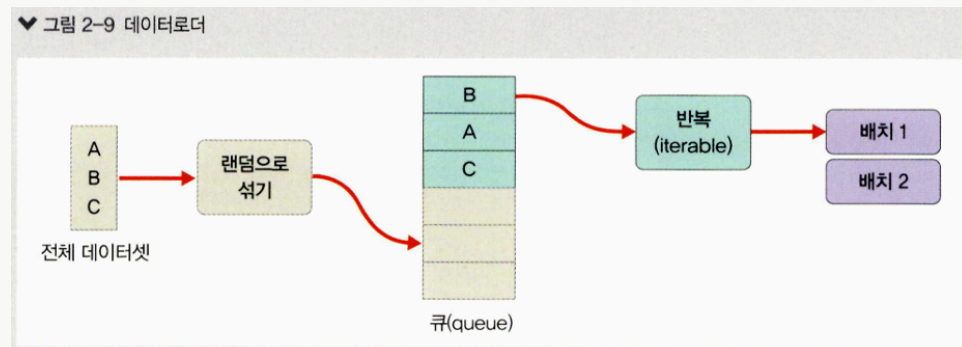
<https://colab.research.google.com/drive/19Rw1xi3FxYjYAYkG-ZGyqVCfkrQLTQVM#scrollTo=BMvCbJdMuZhg>

2.2.1 텐서 다루기

2.2.2 데이터 준비

▼ torch.utils.data.DataLoader

- 데이터로더(DataLoader) 객체는 학습에 사용될 데이터 전체를 보관했다가 모델 학습을 할 때 배치 크기만큼 데이터를 꺼내서 사용
- 데이터를 미리 잘라 놓는 것이 아니라, 내부적으로 반복자(iterator)에 포함된 인덱스(index)를 이용하여 배치 크기만큼 데이터를 반환한다는 것



- 따라서 데이터로더는 다음과 같이 `for` 문을 이용하여 구문을 반복 실행하는 것과 같다

```
for i, data in enumerate(dataset,0):  
    print(i, end='')  
    batch=data[0]  
    print(batch.size())
```

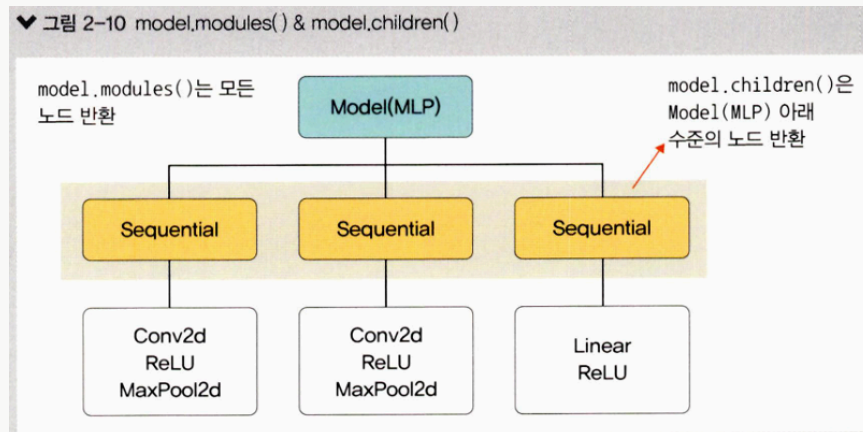
- 다음과 같은 결과가 출력됨

```
0torch.Size([4, 3])  
1torch.Size([4, 3])  
2torch.Size([4, 3])  
3torch.Size([4, 3])  
4torch.Size([4, 3])
```

2.2.3 모델 정의

▼ model.modules() & model.children()

- `model.modules()` 는 모델의 네트워크에 대한 모든 노드를 반환하며, `model.children()` 은 같은 수준(level)의 하위 노드를 반환



2.2.4 모델의 파라미터 정의

• 손실 함수(loss function)

- 학습하는 동안 출력과 실제 값(정답) 사이의 오차를 측정
- 즉, $w x + b$ 를 계산한 값과 실제 값인 y 의 오차를 구해서 모델의 정확성을 측정
 - `BCELoss` : 이진 분류를 위해 사용
 - `CrossEntropyLoss` : 다중 클래스 분류를 위해 사용
 - `MSELoss` : 회귀 모델에서 사용

• 옵티마이저(optimizer)

- 데이터와 손실 함수를 바탕으로 모델의 업데이트 방법을 결정
 - optimizer는 `step()` 메서드를 통해 전달받은 파라미터를 업데이트
 - 모델의 파라미터별로 다른 기준(ex-학습률)을 적용시킬 수 있음
 - `torch.optim.Optimizer(params, defaults)` 는 모든 옵티마이저의 기본이 되는 클래스
 - `zero_grad()` 메서드는 옵티마이저에 사용된 파라미터들의 기울기(gradient)를 0으로 만듦
 - `torch.optim.lr_scheduler` 는 에포크에 따라 학습률을 조절할 수 있음
 - `optim.Adadelta`, `optim.Adagrad`, `optim.Adam`, `optim.SparseAdam`, `optim.Adamax`

- `optim.ASGD`, `optim.LBFGS`
- `optim.RMSProp`, `optim.Rprop`, `optim.SGD`

• 학습률 스케줄러(learning rate scheduler)

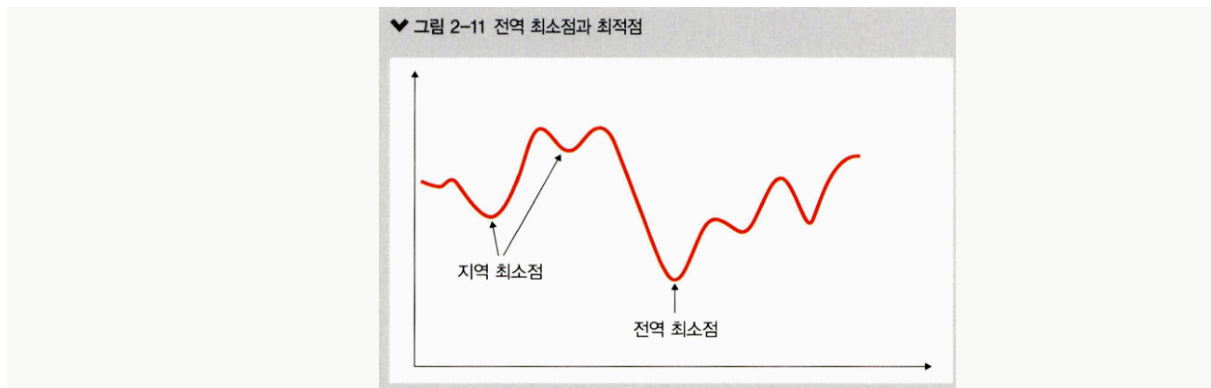
- 미리 지정한 횟수의 에포크를 지날 때마다 학습률을 감소(delay)시켜 줌
- 학습 초기에는 빠른 학습을 진행하다가 전역 최소점(global minimum) 근처에 다르면 학습률을 줄여서 최적점을 찾아갈 수 있도록 해 줌
 - `optim.lr_scheduler.LambdaLR` : 람다 함수를 이용하여 그 함수의 결과를 학습률로 설정
 - `optim.lr_scheduler.StepLR` : 특정 단계마다 학습률을 감마 비율만큼 감소시킴
 - `optim.lr_scheduler.MultiStepLR` : `StepLR` 과 비슷하지만 특정 단계가 아닌 지정된 에포크에만 감마 비율로 감소
 - `optim.lr_scheduler.ExponentialLR` : 에포크마다 이전 학습률에 감마만큼 곱함
 - `optim.lr_scheduler.CosineAnnealingLR` : 학습률을 코사인 함수의 형태처럼 변화시킴, 따라서 학습률이 커지기도 작아지기도 함
 - `optim.lr_scheduler.ReduceLROnPlateau` : 학습이 잘되고 있는지 아닌지에 따라 동적으로 학습률을 변화시킬 수 있음

• 지표(metrics)

- 훈련과 테스트 단계를 모니터링

▼ 전역 최소점과 최적점

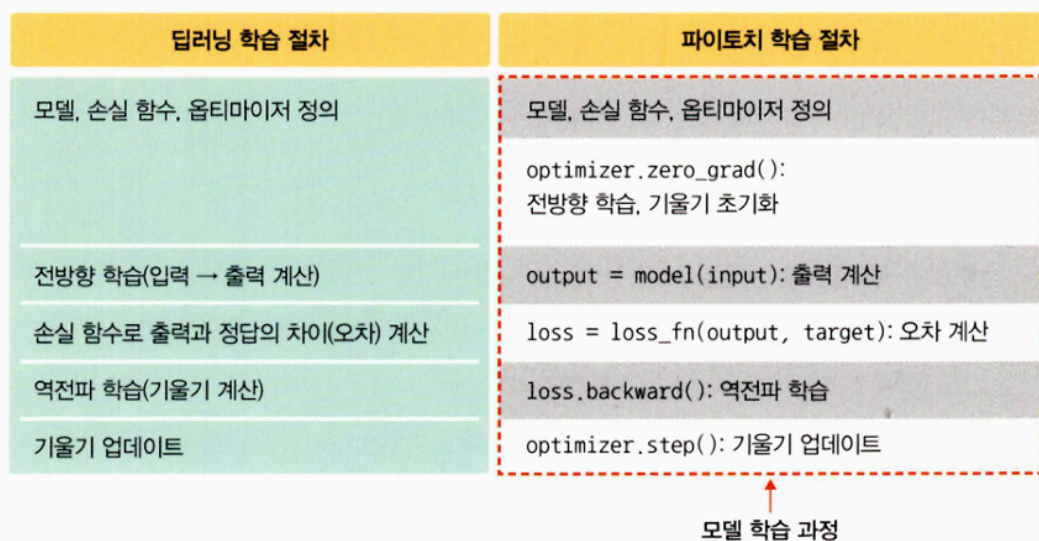
- 학습 목표는 손실 함수의 값을 최소화 하는 가중치와 바이어스를 찾는 것
- 전역 최소점(global minimum)은 오차가 가장 작을 때의 값을 의미, 즉 최적점
- 지역 최소점(local minimum)은 전역 최소점을 찾아가는 과정에서 만나는 홀(hole)과 같은 것으로, 옵티마이저가 지역 최소점에서 학습을 멈추면 최솟값을 갖는 오차를 찾을 수 없는 문제가 발생



2.2.5 모델 훈련

- 앞서 만들어 둔 데이터로 모델을 학습
- 학습을 시킨다는 것은 $y = wx + b$ 라는 함수에서 w 와 b 의 적절한 값을 찾는다는 의미
- w 와 b 에 임의의 값을 적용하여 시작하며 오차가 줄어들어 전역 최소점에 이를 때까지 파라미터(w, b)를 계속 수정
- 가장 먼저 `optimizer.zero_grad()` 메서드를 이용하여 기울기를 초기화
 - 파이토치는 기울기 값을 계산하기 위해 `loss.backward()` 메서드를 이용하는데, 이것을 사용하면 새로운 기울기 값이 이전 기울기 값에 누적하여 계산됨
 - 순환 신경망(Recurrent Neural Network, RNN) 모델을 구현할 때 효과적이지만 누적 계산이 필요하지 않는 모델에 대해서는 불필요
 - 따라서 기울기 값에 대해 누적 계산이 필요하지 않을 때에는 입력 값을 모델에 적용하기 전에 `optimizer.zero_grad()` 메서드를 호출하여 미분 값이 누적되지 않게 초기화해 주어야 함

▼ 그림 2-12 파이토치 학습 절차



- 다음으로 `loss.backward()` 메서드를 이용하여 기울기를 자동 계산
 - `loss.backward()` 는 배치가 반복될 때마다 오차가 중첩적으로 쌓이게 되므로 매번 `zero_grad()` 를 사용하여 미분 값을 0으로 초기화

2.2.6 모델 평가

2.2.7 훈련 과정 모니터링

▼ `model.train()` & `model.eval()`

- `model.train()` : 훈련 데이터셋에 사용하며 모델 훈련이 진행될 것임을 알림, 이때 드롭아웃(dropout)이 활성화됨
- `model.eval()` : 모델을 평가할 때는 모든 노드를 사용하겠다는 의미로 검증과 테스트 데이터셋에 사용
- `model.train()` 과 `model.eval()` 을 선언해야 모델의 정확도를 높일 수 있음
- `model.eval()` 에서 `with torch.no_grad()` 를 사용하는 이유
 - 검증(혹은 테스트) 과정에서는 역전파가 필요하지 않기 때문에 `with torch.no_grad()` 를 사용하여 기울기 값을 저장하지 않도록 함

2.3 실습 환경 설정

2.3.1 아나콘다 설치

2.3.2 가상 환경 생성 및 파이토치 설치

2.4 파이토치 코드 맛보기

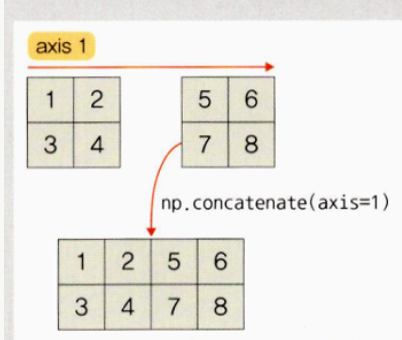
▼ 설치한 라이브러리 설명

- matplotlib: 2D, 3D 형태의 플롯(그래프)을 그릴 때 주로 사용하는 패키지
- seaborn: 데이터 프레임으로 다양한 통계 지표를 표현할 수 있는 시각화 차트를 제공
- scikit-learn: 분류, 회귀, 군집, 의사 결정 트리 등 다양한 머신 러닝 알고리즘을 적용할 수 있는 함수를 제공

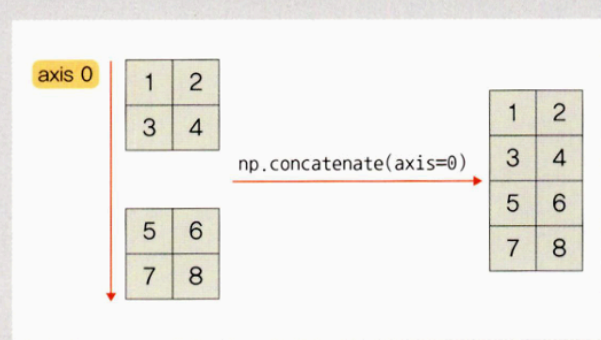
▼ `np.stack`과 `np.concatenate`

- `np.concatenate` 는 선택한 축(axis)을 기준으로 두 개의 배열을 연결

▼ 그림 2-27 `np.concatenate(axis=1)`

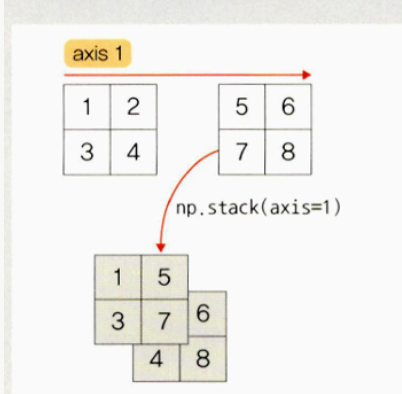


▼ 그림 2-28 `np.concatenate(axis=0)`

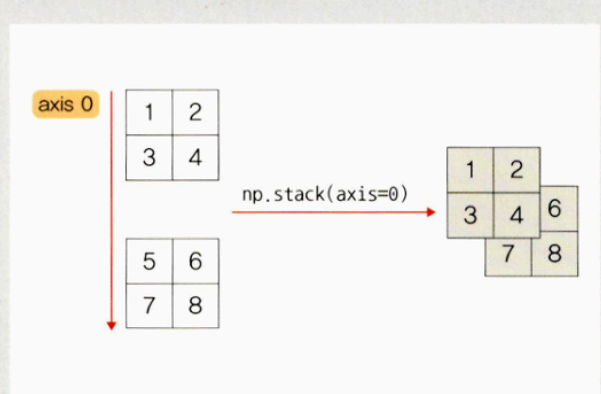


- `np.stack` 은 배열들을 새로운 축으로 합쳐 줌, 따라서 반드시 두 배열의 차원이 동일해야 함

▼ 그림 2-29 `np.stack(axis=1)`



▼ 그림 2-30 `np.stack(axis=0)`



정확도 확인

- **True Positive (TP):** 모델이 1이라고 예측했고 실제 값도 1인 경우
- **True Negative (TN):** 모델이 0이라고 예측했고 실제 값도 0인 경우
- **False Positive (FP):** 모델이 1이라고 예측했는데 실제 값은 0인 경우 (Type I 오류)
- **False Negative (FN):** 모델이 0이라고 예측했는데 실제 값은 1인 경우 (Type II 오류)
- **정확도**
 - 전체 예측 건수에서 정답을 맞춘 비율
 - 맞힌 정답이 Positive인지 Negative인지는 상관 없음)
 - $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$
- **재현율**
 - 실제로 정답이 1일 때, 모델이 1로 맞게 예측한 비율

- 데이터가 1일 확률이 적을 때 사용하면 좋음

- $Recall = \frac{TP}{TP+FN}$

- **정밀도**

- 모델이 1이라고 예측한 것 중 실제로 정답이 1인 비율

- $Precision = \frac{TP}{TP+FP}$

- **F1-스코어**

- 일반적으로 정밀도와 재현율은 트레이드오프 관계
- 정밀도가 높으면 재현율이 낮고, 재현율이 높으면 정밀도가 낮음
- 이를 해결하기 위해 정밀도와 재현율의 조화 평균 이용

- $F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$